

Tempo：免选举的租约定序共识协议（插件演示样例）

张三

2026 年 9 月 26 日

摘要

工程上广泛使用的共识协议，大多把「请求排序」与「状态持久化」这两件事都压在同一个稳定领导者身上：领导者一旦失效，所有写入都要等新一轮选举结束才能恢复。本文提出 Tempo——一种把排序权与持久化权分离的共识协议。排序权由按纪元（epoch）轮转的仲裁节点（arbiter）以短租约（lease）的形式授予，被接受的请求会拿到一枚定序令牌（Ordering Token, OT）；持久化仍然由多数派写入保证。仲裁节点的身份是纪元号的确定性函数，不需要通过选举产生，因此换主只需要一轮仲裁，不存在选举停摆期。在部分同步网络与崩溃失效模型下，我们给出令牌唯一性与纪元单调性两条引理，并据此证明 Tempo 满足线性一致性。

1 引言

共识协议要解决的问题是：让一组可能失效的副本对同一个操作序列达成一致。Paxos [3] 与 Raft [6] 都把「先选出稳定领导者」当作前提，由领导者为请求排序、副本按序追加日志 [4]。这个设计把复杂性收敛到了单一节点，代价是可用性也绑在了这个节点上：领导者失效期间，写入必须停摆

到新一轮选举结束。下面这一行故意引用了一个不存在的 label，用于演示引用校验功能：参见第 ?? 节的动机说明。

本文的贡献可以概括为三点：

- **排序与持久化解耦**：定序由仲裁节点单独授权，写入由多数派确认，两者互不阻塞；
- **免选举换主**：仲裁节点的身份是纪元号的确定性函数，换主不需要任何投票；
- **可分析的快路径**：正常路径 1 RTT，竞争路径 2 RTT，崩溃恢复只做尾部对齐。

2 相关工作与设计动机

租约 (lease) 最早由 Gray 与 Cheriton [2] 提出，用于在分布式文件缓存中避免反复确认；它提供了时间维度上的互斥，但本身不提供共识。Paxos 之后的工程实现 (Multi-Paxos、ZAB、Raft) 选择了「稳定领导者 + 日志复制」的路线，把一轮协商摊薄成常态化的领导权。EPaxos [5] 走向另一个极端：它不设领导者，用依赖集描述命令之间的偏序，冲突稀疏时可以单轮提交，冲突密集时则需要额外的提交轮次。Tempo 借用了「不过度依赖单点」这个动机，但换了一条实现路径：把临时排序权显式化成一枚租约，并让租约携带由多数派确认的纪元栅栏，从而让「排序」这件事可以直接参与安全性论证。

3 系统模型与假设

我们考虑 $n = 2f + 1$ 个副本组成的集群，任意时刻最多 f 个副本失效。失效模型是崩溃失效 (crash-stop)：失效节点不再产生消息，也不做拜占庭行为。网络是部分同步的：存在一个未知时刻 GST，此后任意消息的传输

延迟不超过已知上界 Δ 。多数派 (quorum) 定义为任意 $\lceil (n+1)/2 \rceil$ 个副本。以下三点假设贯穿全文：

1. 副本的本地时钟只要求单调不回退，不要求彼此同步；
2. 客户端允许重试，因此协议只需要保证「至多生效一次」的语义；
3. 消息可能丢失、重复与乱序，但不会损坏，也不会被篡改。

4 协议设计

Tempo 把一次写入拆成两个可以独立推进的阶段：先向仲裁节点取定顺序令牌，再把请求按令牌顺序写入多数派。图 1 给出了正常路径上的消息流。

图 1: 正常路径的消息流：客户端 $\rightarrow$ 提案节点 $\rightarrow$ 仲裁节点 $\rightarrow$ 多数派

4.1 混合逻辑时钟与纪元栅栏

副本维护一个混合逻辑时钟 (Hybrid Logical Clock, HLC) [1]，它取本地物理时间与收到的最大逻辑时间中的较大者，只要求单调不回退。纪元 (epoch) 是排序权的时间切片。当仲裁节点需要开启新纪元 $e+1$ 时，它先向 $n-1$ 个副本发出探测，收集其中 $\lceil (n+1)/2 \rceil - 1$ 个确认，再按下式推进基准时钟：

$$t_{e+1} = \max(t_e, h_{\max}) + 1 \quad (1)$$

其中 $h_{\max}$ 是确认集合 $\mathcal{A}$ 中各副本报告的 HLC 的最大值。式 (1) 得到的 t_{e+1} 称为纪元栅栏：任何属于纪元 $e+1$ 的请求，其时间戳都不允许小于这道栅栏。

4.2 定序令牌与租约

仲裁节点在纪元 e 内为本地的序号 s 单调递增地签发定序令牌：

$$\text{OT} = \langle e, s \rangle, \quad \langle e, s \rangle < \langle e', s' \rangle \Leftrightarrow e < e' \vee (e = e' \wedge s < s') \quad (2)$$

令牌之间按字典序 (lexicographic order) 比较, 因此任意两枚令牌都可以比出大小, 这也让提交序列在副本之间有了唯一确定的排列。一枚令牌只在租约期内有效: 租约时长 L 需要在「仲裁节点的负载」与「失效检测的迟缓」之间折中, 工程上通常取 L 为几倍的消息往返时间。¹ 令牌有效当且仅当:

$$\text{valid}(\text{OT}) \Leftrightarrow \text{now} < \text{expire}(\text{OT}) \wedge \text{Arb}(e) = a \wedge |\text{Ack}_e| \geq \lceil (n+1)/2 \rceil \quad (3)$$

其中 $\text{Arb}(e)$ 是由纪元号唯一确定的仲裁节点, Ack_e 是纪元 e 收集到的确认集合。

4.3 正常路径与竞争路径

正常路径上, 提案节点把请求连同租约一起广播给多数派, 收到多数派的写入确认后即可提交, 总耗时为 1 RTT。若租约已经过期, 或者仲裁节点在探测阶段没能在 L 内收到足够确认, 请求就转入竞争路径: 提案节点重新收集 HLC、推进纪元栅栏, 再取一枚新令牌, 总耗时为 2 RTT。两条路径的差别只体现在延迟上, 不影响安全性。

5 正确性论证

本节给出 Tempo 的安全性论证。记号约定: OT_x 表示请求 x 的定序令牌, Ack_e 表示纪元 e 中收集到确认的副本集合, t_e 表示纪元 e 的基准时钟。

¹本文取 $L = 3\Delta$, 并在实现中把它做成可配置项。

引理 1 (令牌唯一性) 同一纪元内, 仲裁节点不会把同一个序号签发给两个不同的请求。证明: 仲裁节点对序号 s 的分配是本地单调递增的, 而且签发之前必须通过租约有效性检查 (3); 序号一旦被签发, 单调性使得后续请求只能拿到更大的序号。证毕。

引理 2 (纪元单调性) 若纪元 $e' > e$, 则 $t_{e'} > t_e$ 。证明: 推进纪元需要收集 $\lceil (n+1)/2 \rceil - 1$ 个副本的确认, 并按式 (1) 取最大 HLC 再加一。两次推进的确认集合大小之和超过 n , 因此必存在一个副本同时出现在两次确认集合里; 该副本的 HLC 单调不回退, 于是后一次推进读到的最大值不小于前一次, 即 $t_{e'} > t_e$ 。证毕。

定理 1 (线性一致性) 在部分同步网络中, Tempo 的提交序列等价于某个顺序执行。证明: 由引理 1, 同一纪元内不存在两枚相同的令牌; 由引理 2, 纪元之间不会出现倒序。于是全部提交都可以按式 (2) 的字典序排列成一个唯一的全序。每次提交都写入了多数派, 任意两次提交的写入集合相交, 交集中的副本按同一顺序追加请求, 因此任一时刻任一读取看到的都是这个全序的前缀。证毕。

6 对比与评估

表 1 给出四类共识协议在 5 节点集群上的开销对比。

表 1: 四类共识协议在 5 节点集群上的开销对比

协议	需领导者	快路径 RTT	换主	恢复
Multi-Paxos	是	1	选举 · 随机退避	补齐空洞
Raft	是	1	选举 · 随机超时	截断冲突
EPaxos	否	1-2	无需换主	依赖集重放
Tempo (本文)	否	1	一轮仲裁	尾部对齐

Multi-Paxos 与 Raft 在领导者稳定时都能达到 1 RTT, 但它们把可用性绑定在领导者身上: 领导者失效之后, 集群要等到选举超时 (通常是 150 ~ 300 ms) 才能恢复服务。Tempo 的仲裁轮只需要一次往返, 而且在

△ 有界的假设下是可以预期的：换主不需要任何节点就「谁是新仲裁者」达成共识，因为新仲裁者由纪元号直接算出。EPaxos 同样不需要领导者，但它的事务提交流程与命令之间的依赖关系有关，冲突密集时延迟会上升；Tempo 把排序权收敛到单枚租约上，因此路径长度与冲突密度无关。代价是仲裁节点在租约期内会成为热点。

7 局限与未来工作

本文只讨论了单组 (single-shard) 场景。跨组事务需要额外引入协调者，这会重新带回 Tempo 试图消除的那个单点。另一处局限是纪元栅栏依赖时钟的单调性：如果某个副本发生时钟回退，栅栏必须显式拒绝它的确认。未来工作包括把租约时长做成自适应参数，并在更大规模的集群上补做实验。

8 结论

本文提出了 Tempo：一种把排序权与持久化权分离的共识协议。它用一枚由多数派确认过的定序令牌替代了领导者选举，使换主成为一次有界的仲裁轮。我们给出了令牌唯一性与纪元单调性两条引理，并据此证明了线性一致性。在正常路径上，Tempo 与 Raft 具有相同的往返次数，但换主过程不再需要选举停摆期。

参考文献

- [1] Lamport L. Time, clocks, and the ordering of events in a distributed system. Communications of the ACM, 1978.
- [2] Gray C, Cheriton D. Leases: an efficient fault-tolerant mechanism for distributed file cache consistency. SOSP, 1989.

- [3] Lamport L. The part-time parliament. ACM Transactions on Computer Systems, 1998.
- [4] Lamport L. Paxos made simple. ACM SIGACT News, 2001.
- [5] Moraru I, Andersen D G, Kaminsky M. There is more consensus in egalitarian parliaments. SOSP, 2013.
- [6] Ongaro D, Ousterhout J. In search of an understandable consensus algorithm. USENIX ATC, 2014.